

Guía de Desarrollo de un Rogue-like Simple en Greenfoot

Drivers arquitectónicos, memoria, rendimiento, modularidad y dificultad progresiva

Propósito del laboratorio

En esta actividad se construirá un pequeño videojuego tipo *rogue-like* de supervivencia. El jugador controla un soldado que debe resistir oleadas de soldados enemigos. La dificultad aumentará gradualmente, pero el sistema deberá impedir que el crecimiento de enemigos y proyectiles produzca un consumo descontrolado de memoria o una degradación severa del rendimiento.

1. Resultado esperado

El prototipo utilizará figuras simples generadas mediante código. El jugador podrá moverse con WASD, disparar con la barra espaciadora, recibir daño, eliminar enemigos y avanzar por oleadas. Los enemigos aparecerán en los bordes, perseguirán al jugador y aumentarán gradualmente en cantidad y resistencia.

Jugador → combate → oleada → supervivencia → oleada más difícil

2. Primero: pensar la arquitectura

La lógica de razonamiento será:

Drivers → Concerns → Tensiones → Decisiones → Código

3. Drivers arquitectónicos

Driver	Necesidad del juego	Impacto arquitectónico
Jugabilidad	Respuesta inmediata al movimiento y disparo.	Las entradas deben procesarse independientemente de la lógica de oleadas.
Rendimiento	Mantener ejecución fluida al aumentar enemigos.	Limitar actores simultáneos y trabajo por <code>act()</code> .
Memoria	Cada enemigo, bala e imagen ocupa memoria.	Eliminar actores muertos, limitar balas y reutilizar imágenes.
Escalabilidad de dificultad	Cada oleada debe ser más desafiante.	Aumentar dificultad mediante parámetros, no mediante actores ilimitados.
Modularidad	Jugador, enemigo, proyectil, oleadas e interfaz deben evolucionar separadamente.	Separar responsabilidades en clases.
Mantenibilidad	Agregar tipos de enemigos no debe obligar a reescribir el juego.	Centralizar comportamiento común en Soldado .
Observabilidad	El estudiante debe poder ver cuántos actores relevantes existen.	Incorporar HUD de oleada, vida, enemigos y balas.

4. Concerns principales

- **Responsiveness:** respuesta inmediata.
- **Performance:** trabajo razonable por frame.
- **Memory usage:** evitar acumulación de actores y recursos.
- **Modularity:** separar responsabilidades.
- **Scalability:** aumentar dificultad de forma controlada.
- **Maintainability:** facilitar cambios.
- **Game balance:** dificultad creciente pero jugable.
- **Observability:** poder inspeccionar el estado del juego.
- **Resource lifecycle:** crear y destruir objetos en momentos definidos.

5. Tensión fundamental: memoria versus rendimiento

Si existen N_e enemigos y N_b proyectiles, una aproximación conceptual del trabajo por ciclo es:

$$C_{frame} \approx C_0 + N_e C_e + N_b C_b$$

El uso de memoria puede aproximarse por:

$$M_{total} \approx M_0 + N_e M_e + N_b M_b + M_{recursos}$$

Por tanto:

más enemigos = más presión de juego + más memoria + más trabajo por frame

La dificultad no puede crecer simplemente como $10 \rightarrow 50 \rightarrow 100 \rightarrow 500$ enemigos simultáneos.

6. Dificultad versus estabilidad

Se separan dos conceptos:

enemigos totales de la oleada $\neq$ enemigos simultáneos

Una oleada puede contener muchos enemigos, pero solo una parte de ellos necesita estar activa al mismo tiempo.

Oleada	Total	Máx. simultáneos	Vida aprox.
1	8	8	24
2	11	10	28
3	14	12	32
4	17	14	36
5	20	16	40

7. Memoria versus reutilización

No conviene crear una nueva imagen en cada `act()`. Para figuras estáticas se adopta:

crear una vez $\rightarrow$ reutilizar muchas veces

8. Modularidad versus simplicidad

Todo podría implementarse en una clase, pero eso mezclaría responsabilidades. La estructura será:

```
JuegoWorld
|
+-- GestorOleadas
+-- HUD
+-- Soldado
    |
    +-- Jugador
    +-- SoldadoEnemigo
```

```
Bala
FabricaImagenes
```

9. Decisiones arquitectónicas

1. limitar enemigos simultáneos;
2. limitar cantidad y vida útil de proyectiles;
3. eliminar actores derrotados;
4. reutilizar imágenes estáticas;
5. separar oleadas del comportamiento enemigo;
6. centralizar comportamiento común en **Soldado**;
7. aumentar dificultad mediante varios parámetros;
8. actualizar el HUD con menor frecuencia que la lógica del juego.

10. Arquitectura del juego

Clase	Responsabilidad
JuegoWorld	Contener el mundo, iniciar componentes y gestionar game over.
Soldado	Vida, velocidad y comportamiento común.
Jugador	Teclado, movimiento y disparo.
SoldadoEnemigo	Persecución y ataque.
Bala	Movimiento, impacto y ciclo de vida.
GestorOleadas	Creación de enemigos y escalamiento de dificultad.
HUD	Información de ejecución.
FabricaImágenes	Creación y reutilización de figuras.

11. Paso 1: crear el escenario

```
import greenfoot.*;

public class JuegoWorld extends World
{
    private Jugador jugador;
    private GestorOleadas gestor;
    private HUD hud;

    public JuegoWorld()
    {
        super(900, 600, 1);

        jugador = new Jugador();
        addObject(jugador, getWidth()/2, getHeight()/2);

        gestor = new GestorOleadas(this, jugador);

        hud = new HUD(this, jugador, gestor);
        addObject(hud, 220, 25);
    }

    public void act()
    {
        gestor.actualizar();
    }

    public void gameOver()
    {
        showText("GAME OVER", getWidth()/2, getHeight()/2);
        Greenfoot.stop();
    }
}
```

12. Paso 2: crear figuras reutilizables

```
import greenfoot.*;

public class FabricaImagenes
{
    private static final GreenfootImage JUGADOR =
        crearSoldado(Color.GREEN);

    private static final GreenfootImage ENEMIGO =
        crearSoldado(Color.RED);

    private static final GreenfootImage BALA = crearBala();

    public static GreenfootImage soldadoJugador()
    {
        return JUGADOR;
    }

    public static GreenfootImage soldadoEnemigo()
    {
        return ENEMIGO;
    }

    public static GreenfootImage bala()
    {
        return BALA;
    }

    private static GreenfootImage crearBala()
    {
        GreenfootImage img = new GreenfootImage(10, 4);
        img.setColor(Color.YELLOW);
        img.fillRect(0, 0, 10, 4);
        return img;
    }

    private static GreenfootImage crearSoldado(Color color)
    {
        GreenfootImage img = new GreenfootImage(34, 34);
        img.setColor(color);
        img.fillOval(11, 1, 12, 12);    // cabeza
        img.fillRect(10, 12, 14, 15);  // cuerpo
        img.fillRect(4, 14, 7, 5);      // brazo
        img.fillRect(23, 14, 7, 5);     // brazo
        img.fillRect(10, 26, 5, 8);     // pierna
        img.fillRect(19, 26, 5, 8);     // pierna
        return img;
    }
}
```

Así, una misma imagen puede ser utilizada por muchos actores enemigos.

13. Paso 3: crear una clase base Soldado

```
import greenfoot.*;

public abstract class Soldado extends Actor
{
    protected int vida;
    protected int velocidad;

    public Soldado(int vida, int velocidad)
    {
        this.vida = vida;
        this.velocidad = velocidad;
    }

    public void recibirDanio(int cantidad)
    {
        vida -= cantidad;
        if (vida <= 0)
        {
            morir();
        }
    }

    public int getVida()
    {
        return vida;
    }

    protected void morir()
    {
        World mundo = getWorld();
        if (mundo != null)
        {
            mundo.removeObject(this);
        }
    }
}
```

14. Paso 4: implementar el jugador

```
import greenfoot.*;

public class Jugador extends Soldado
{
    private int cooldownDisparo = 0;
    private int dirX = 1;
    private int dirY = 0;

    public Jugador()
    {
        super(100, 4);
        setImage(FabricaImagenes.soldadoJugador());
    }

    public void act()
    {
        mover();
        disparar();
        if (cooldownDisparo > 0) cooldownDisparo--;
    }

    private void mover()
    {
        int dx = 0;
        int dy = 0;

        if (Greenfoot.isKeyDown("w"))
        {
            dy = -velocidad;
            dirX = 0; dirY = -1;
        }
        if (Greenfoot.isKeyDown("s"))
        {
            dy = velocidad;
            dirX = 0; dirY = 1;
        }
        if (Greenfoot.isKeyDown("a"))
        {
            dx = -velocidad;
            dirX = -1; dirY = 0;
        }
        if (Greenfoot.isKeyDown("d"))
        {
            dx = velocidad;
            dirX = 1; dirY = 0;
        }

        int nuevoX = Math.max(15,
            Math.min(getWorld().getWidth()-15, getX()+dx));
        int nuevoY = Math.max(15,
            Math.min(getWorld().getHeight()-15, getY()+dy));

        setLocation(nuevoX, nuevoY);
    }

    private void disparar()
    {
        if (Greenfoot.isKeyDown("space") && cooldownDisparo == 0)
```



```
{
    int cantidadBalas = getWorld().getObjects(Bala.class).size();

    if (cantidadBalas < 60)
    {
        Bala bala = new Bala(dirX, dirY);
        getWorld().addObject(bala, getX(), getY());
    }
    cooldownDisparo = 8;
}

protected void morir()
{
    World mundo = getWorld();
    if (mundo instanceof JuegoWorld)
    {
        ((JuegoWorld)mundo).gameOver();
    }
    if (mundo != null) mundo.removeObject(this);
}
}
```

Se establece explícitamente el presupuesto:

$$N_b \leq 60$$

15. Paso 5: crear el proyectil

```
import greenfoot.*;

public class Bala extends Actor
{
    private int dx;
    private int dy;
    private int velocidad = 8;
    private int vidaUtil = 80;

    public Bala(int dirX, int dirY)
    {
        dx = dirX;
        dy = dirY;
        setImage(FabricaImagenes.bala());

        if (dx == 1) setRotation(0);
        if (dx == -1) setRotation(180);
        if (dy == 1) setRotation(90);
        if (dy == -1) setRotation(270);
    }

    public void act()
    {
        setLocation(getX()+dx*velocidad, getY()+dy*velocidad);
        vidaUtil--;

        SoldadoEnemigo enemigo = (SoldadoEnemigo)
            getOneIntersectingObject(SoldadoEnemigo.class);

        if (enemigo != null)
        {
            enemigo.recibirDanio(10);
            getWorld().removeObject(this);
            return;
        }

        if (vidaUtil <= 0 || fueraDelMundo())
        {
            getWorld().removeObject(this);
        }
    }

    private boolean fueraDelMundo()
    {
        return getX() <= 2 || getY() <= 2
            || getX() >= getWorld().getWidth()-2
            || getY() >= getWorld().getHeight()-2;
    }
}
```

La bala posee un ciclo de vida explícito:

crear → usar → destruir

16. Paso 6: implementar el enemigo

```
import greenfoot.*;

public class SoldadoEnemigo extends Soldado
{
    private Jugador objetivo;
    private int danio;
    private int cooldownAtaque = 0;

    public SoldadoEnemigo(Jugador objetivo,
                          int vida,
                          int velocidad,
                          int danio)
    {
        super(vida, velocidad);
        this.objetivo = objetivo;
        this.danio = danio;
        setImage(FabricaImagenes.soldadoEnemigo());
    }

    public void act()
    {
        if (objetivo == null || objetivo.getWorld() == null) return;

        perseguir();
        if (cooldownAtaque > 0) cooldownAtaque--;
        atacar();
    }

    private void perseguir()
    {
        int dx = objetivo.getX() - getX();
        int dy = objetivo.getY() - getY();
        double distancia = Math.sqrt(dx*dx + dy*dy);

        if (distancia > 0)
        {
            int movX = (int)Math.round(velocidad * dx / distancia);
            int movY = (int)Math.round(velocidad * dy / distancia);
            setLocation(getX()+movX, getY()+movY);
        }
    }

    private void atacar()
    {
        if (isTouching(Jugador.class) && cooldownAtaque == 0)
        {
            objetivo.recibirDanio(danio);
            cooldownAtaque = 30;
        }
    }
}
```

El enemigo no conoce la oleada ni decide cuántos enemigos deben existir. Esa responsabilidad pertenece al gestor de oleadas.

17. Paso 7: crear el gestor de oleadas

```
import greenfoot.*;

public class GestorOleadas
{
    private JuegoWorld mundo;
    private Jugador jugador;
    private int oleada = 1;
    private int generados = 0;
    private int objetivoOleada = 0;
    private int contadorSpawn = 0;
    private int esperaEntreOleadas = 0;

    public GestorOleadas(JuegoWorld mundo, Jugador jugador)
    {
        this.mundo = mundo;
        this.jugador = jugador;
        prepararOleada();
    }

    public void actualizar()
    {
        if (jugador.getWorld() == null) return;

        int activos = mundo.getObjects(SoldadoEnemigo.class).size();

        if (generados < objetivoOleada)
        {
            contadorSpawn--;

            if (contadorSpawn <= 0 && activos < maxSimultaneos())
            {
                crearEnemigo();
                generados++;
                contadorSpawn = intervaloSpawn();
            }
        }
        else if (activos == 0)
        {
            if (esperaEntreOleadas == 0) esperaEntreOleadas = 100;

            esperaEntreOleadas--;

            if (esperaEntreOleadas <= 0)
            {
                oleada++;
                prepararOleada();
            }
        }
    }

    private void prepararOleada()
    {
        generados = 0;
        objetivoOleada = Math.min(5 + oleada*3, 80);
        contadorSpawn = 20;
        esperaEntreOleadas = 0;
    }
}
```

```
private int maxSimultaneos()
{
    return Math.min(6 + oleada*2, 35);
}

private int intervaloSpawn()
{
    return Math.max(12, 50 - oleada*3);
}

private int vidaEnemigo()
{
    return 20 + oleada*4;
}

private int velocidadEnemigo()
{
    return Math.min(1 + oleada/4, 4);
}

private int danioEnemigo()
{
    return Math.min(4 + oleada, 15);
}

private void crearEnemigo()
{
    SoldadoEnemigo enemigo = new SoldadoEnemigo(
        jugador,
        vidaEnemigo(),
        velocidadEnemigo(),
        danioEnemigo());

    int lado = Greenfoot.getRandomNumber(4);
    int x = 0;
    int y = 0;

    if (lado == 0)
    {
        x = 10;
        y = Greenfoot.getRandomNumber(mundo.getHeight());
    }
    else if (lado == 1)
    {
        x = mundo.getWidth() - 10;
        y = Greenfoot.getRandomNumber(mundo.getHeight());
    }
    else if (lado == 2)
    {
        x = Greenfoot.getRandomNumber(mundo.getWidth());
        y = 10;
    }
    else
    {
        x = Greenfoot.getRandomNumber(mundo.getWidth());
        y = mundo.getHeight() - 10;
    }

    mundo.addObject(enemigo, x, y);
}
```

```
public int getOleada()  
{  
    return oleada;  
}
```

18. Paso 8: comprender la política de dificultad

La cantidad total de enemigos es:

$$N_{oleada} = \text{mín}(5 + 3w, 80)$$

El máximo simultáneo es:

$$N_{sim} = \text{mín}(6 + 2w, 35)$$

por tanto:

$$\boxed{N_{sim} \leq 35}$$

La vida crece como:

$$HP(w) = 20 + 4w$$

y el intervalo de aparición como:

$$I(w) = \text{máx}(12, 50 - 3w)$$

La dificultad combina cantidad, vida, frecuencia de aparición, velocidad y daño. No depende exclusivamente de N .

19. Paso 9: incorporar un HUD

```
import greenfoot.*;

public class HUD extends Actor
{
    private JuegoWorld mundo;
    private Jugador jugador;
    private GestorOleadas gestor;
    private int contador = 0;

    public HUD(JuegoWorld mundo,
               Jugador jugador,
               GestorOleadas gestor)
    {
        this.mundo = mundo;
        this.jugador = jugador;
        this.gestor = gestor;
        actualizarImagen();
    }

    public void act()
    {
        contador++;
        if (contador >= 10)
        {
            actualizarImagen();
            contador = 0;
        }
    }

    private void actualizarImagen()
    {
        int enemigos = mundo.getObjects(SoldadoEnemigo.class).size();
        int balas = mundo.getObjects(Bala.class).size();

        String texto = "Oleada: " + gestor.getOleada()
            + " | Vida: " + jugador.getVida()
            + " | Enemigos: " + enemigos
            + " | Balas: " + balas;

        GreenfootImage img = new GreenfootImage(
            texto, 22, Color.WHITE, Color.BLACK);
        setImage(img);
    }
}
```

El HUD se actualiza cada 10 ciclos porque su información no necesita recrearse en cada frame.

20. Paso 10: probar la primera versión

Antes de escalar, registrar:

- máximo de enemigos simultáneos;
- máximo de balas;
- respuesta del jugador;

- si las balas desaparecen;
- si los enemigos muertos son eliminados;
- si la vida disminuye correctamente.

21. Paso 11: experimentar con crecimiento sin límites

Como experimento controlado, cambiar temporalmente el máximo simultáneo por 500 y observar la respuesta del teclado y la fluidez. Luego restaurar el límite.

Pregunta arquitectónica

¿El problema se resuelve optimizando solamente la clase `SoldadoEnemigo`, o también necesitamos controlar cuántos enemigos permitimos que existan simultáneamente?

La respuesta esperada es que ambas decisiones importan.

22. Paso 12: analizar la cantidad de actores

Con los límites definidos:

$$N_e \leq 35, \quad N_b \leq 60$$

Por ello, en condiciones normales el número de actores relevantes permanece cercano a un centenar, en lugar de crecer indefinidamente.

23. Paso 13: progresión de dificultad

Oleada	Total	Simult.	HP	Spawn	Vel.
1	8	8	24	47	1
2	11	10	28	44	1
3	14	12	32	41	1
4	17	14	36	38	2
5	20	16	40	35	2
8	29	22	52	26	3
10	35	26	60	20	3
15	50	35	80	12	4

Cuando $N_{sim} = 35$, la cantidad deja de crecer, pero la dificultad puede seguir aumentando mediante otros parámetros.

24. Paso 14: introducir tipos de enemigos

La arquitectura permite evolucionar hacia:

```
SoldadoEnemigo
|
+-- EnemigoRapido
+-- EnemigoPesado
+-- EnemigoElite
```


Por ejemplo:

```
public class EnemigoPesado extends SoldadoEnemigo
{
    public EnemigoPesado(Jugador jugador)
    {
        super(jugador, 100, 1, 12);
    }
}
```

Esto permite aumentar dificultad mediante composición de amenazas y no solamente mediante cantidad.

25. Paso 15: introducir variabilidad rogue-like

Una oleada puede elegir una variante aleatoria:

```
int variante = Greenfoot.getRandomNumber(3);

if (variante == 0)
{
    // mas enemigos, menor vida
}
else if (variante == 1)
{
    // menos enemigos, mas resistentes
}
else
{
    // aparicion mas rapida
}
```

Así:

variabilidad → rejugabilidad

sin necesidad de aumentar indefinidamente la cantidad de actores.

26. Errores que deben evitarse

1. generar enemigos en cada `act()` sin límite;
2. crear proyectiles que nunca son eliminados;
3. recrear imágenes estáticas continuamente;
4. mezclar la lógica de oleadas con el comportamiento individual del enemigo;
5. usar cantidad como único mecanismo de dificultad.

27. Experimentos de arquitectura

Experimento A: cantidad máxima de enemigos

Probar $N_{max} = 10, 20, 35, 50$ y registrar dificultad, respuesta y fluidez.

Experimento B: cantidad máxima de proyectiles

Comparar $B_{max} = 20, 60, 150$. Preguntarse si 150 proyectiles aportan realmente valor de jugabilidad.

Experimento C: vida útil

Comparar `vidaUtil=40` y `vidaUtil=200`.

Experimento D: dificultad sin más actores

Mantener $N_{max} = 20$ y aumentar HP, daño, velocidad y frecuencia de aparición.

28. Actividad de razonamiento arquitectónico

Concern	Riesgo	Decisión	Código
Memoria			
Rendimiento			
Modularidad			
Escalabilidad			
Jugabilidad			
Observabilidad			

29. Preguntas de análisis

1. ¿Por qué una oleada de 80 enemigos no requiere 80 enemigos simultáneos?
2. ¿Qué ocurre con el costo por frame cuando aumenta N_e ?
3. ¿Por qué las balas necesitan una vida útil?
4. ¿Qué ventaja entrega compartir imágenes?
5. ¿Qué responsabilidad cumple `GestorOleadas`?
6. ¿Por qué el enemigo no debería conocer el número de oleada?
7. ¿Qué concern se protege mediante el límite de 35 enemigos?
8. ¿Qué otras variables aumentan dificultad sin aumentar actores?
9. ¿Qué cambiaría si los enemigos también dispararan?
10. ¿Cuándo sería razonable considerar object pooling?

30. Extensión opcional: object pooling

En una escala mayor puede aparecer la tensión entre crear/destruir objetos y reutilizarlos. Un *object pool* sigue conceptualmente:

crear $\rightarrow$ usar $\rightarrow$ desactivar $\rightarrow$ reutilizar

No es necesario implementarlo en esta primera versión; el objetivo es reconocer que la decisión depende de la escala observada.

31. Arquitectura resultante

La solución mantiene presupuestos explícitos:

$$N_e \leq 35 \quad N_b \leq 60 \quad TTL_b \leq 80$$

La arquitectura, por tanto, no organiza solamente clases: también define límites, ciclos de vida, responsabilidades, reglas de crecimiento y presupuestos de recursos.

32. Conclusión

El laboratorio muestra que:

$$\text{más dificultad} \not\Rightarrow \text{más objetos sin límite}$$

La dificultad creciente presiona memoria y rendimiento. La respuesta arquitectónica combina:

$$\text{modularidad} + \text{límites de concurrencia} + \text{ciclo de vida} + \text{reutilización} + \text{dificultad multiparámetro}$$

El resultado es un rogue-like sencillo que sirve simultáneamente como ejercicio de programación orientada a objetos y como introducción práctica al razonamiento arquitectónico basado en drivers, concerns y trade-offs.